# INTRODUCCIÓ ALS ORDINADORS

## INFORME PRÀCTICA 6

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

## INTRODUCCIÓ

En aquesta pràctica es fa una presentació de la memòria caché, una memòria que ens aporta velocitat i rendiment en l'ordinador en el moment de fer les tasques, ja que és una memòria RAM estàtica. Per altra banda, s'aprofundirà en l'ús del simulador i8085, fent ús de subrutines i displays de 7 segments, LEDS, així com pantalles de text. També s'aprofundirà en el comportament de la pila.

## OBJECTIUS

Com a objectius de la pràctica es té estudiar l'espai de memòries del microprocessador, comprendre el funcionament de la pila, així com les seves modificacions. A més a més també es té com a objectiu veure exemples d'utilització de subrutines en diferents programes, tant en Ripes com en i8085.

### Cos de la Part I (Ripes)

**El següent codi calcula a partir de un determinat valor (10 per defecte) el factorial i la sèrie deFibonacci, guardant tant el resultat del factorial com els diferents valors de la sèrie en memòria.**

```
.data
valor: .word 10
factorial: .word 0
.text
.globl main
main:
la a1, valor
lw a2, 0(a1)
li a3, 1
ble a2, a3, final
call subrutinaFactorial
call subrutinaFibonacci
final:
```

```
sw a4, 4(a1)
while_true:
nop
j while_true
.text
subrutinaFactorial:
addi sp, sp, -8
sw a2, 0(sp)
sw a3, 4(sp)
loopFac:
beqz a2, end
mul a3, a2, a3
bltz a3, negatiu
addi a2,a2,-1
j loopFac
negatiu:
li a3, -1
end:
mv a4, a3 # Guardem el valor del factorial en a4
lw a3, 4(sp) # Recuperem els valors de a3
lw a2, 0(sp) # Recuperem el valor de a2
ret # Recuperem el PC i tornem
.text
subrutinaFibonacci:
addi sp, sp, -12
sw a1, 0(sp)
sw a2, 4(sp)
sw a3, 8(sp)
sw a4, 12(sp)
lw a2,0(a1)
li a3, 0
li a4, 1
```

```
li a0, 0
beq a2,a3 endFib
sw a3, 0x50(a1)
addi a0,a0,1
beq a0,a2 endFib
addi a1, a1, 4
sw a4, 0x50(a1)
addi a0,a0,1
loopFib:
beq a0,a2 endFib
add a5,a4,a3
mv a3,a4
mv a4,a5
addi a1,a1,4
sw a5,0x50(a1)
addi a0,a0,1
j loopFib
endFib:
lw a4,12(sp)
lw a3,8(sp)
lw a2,4(sp)
lw a1,0(sp)
ret
```

**i) Analitza el funcionament de la cache del simulador. Explica primer com es va omplint la cache i com evolucionen les errades i els encerts en funció de l'evolució del programa. Explica com evoluciona en funció de si el tipus de cache és 4-word direct maped, 4-word 2-way set associative o**
**4-word fully associative. Un cop executat tot el codi, feu una taula on es mostri, en funció del tipus de cache, la tassa d'encerts (Hit rate), els encerts (Hits) i les errades (Misses), tant per la cache de dades com per la cache de instruccions. Explica el resultat, tant per Write-through com per Writeback.**

La memòria caché és un tipus de memòria jeràrquica, concretament una memòria RAM estàtica. Aquesta memòria es troba entre la CPU i la memòria principal de l'ordinador, que és la memòria RAM dinàmica. L'objectiu de la memòria caché és reduir el temps d'accés a les dades més utilitzades en el programa. Per tant, el que fa aquesta memòria és emmagatzemar de forma temporal, durant l'execució del programa, les dades que s'utilitzen amb més freqüència. Per tant, quan el processador requereix accedir a una adreça de memòria per tal d'obtenir una dada, es comprova a la memòria caché si aquesta adreça està allà. En cas que estigui a la memòria caché, aleshores es produeix una lectura ràpida i eficient, que és l'objectiu d'aquesta memòria. En cas contrari, s'haurà d'accedir a la memòria principal per tal d'obtenir aquesta dada, que és un procés més lent i costós. En el cas de Ripes, en l'apartat on podem veure el funcionament de la caché, veiem que hi ha la caché de les dades i de les instruccions. En particular, en cada finestra es pot comprovar com està funcionant aquesta, juntament amb les dades de Hit o Miss i les estadístiques.

Tal i com s'ha vist en l'explicació teòrica de la pràctica, a més a més d'observar el seu funcionament al Ripes, s'ha pogut confirmar el que es deia a teoria. En particular, en el cas del tipus de cache és 4-word direct mapped, es pot veure que a mesura que el processador va executant el programa va guardant les adreces corresponents en el lloc de la caché corresponent. En el cas de parts del codi en les que encara no s'ha accedit ( per exemple blocs de codi on es pot accedir una vegada es fa un salt condicional o cridar a subrutines), es poden carregar a la caché, però no s'accedirà. I en cas que s'accedeixi a una adreça corresponent a una instrucció que és posterior al codi, aleshores en l'espai de la caché on anirien les instruccions anteriors es deixa en blanc, fins que sigui el moment d'accedir per primera vegada. Per tant, la forma en la que s'omple la caché segueix l’ordre d'execució del programa, però l'ordre que segueix la memòria caché és el mateix que segueix l'ordre de les instruccions al programa.

Per altra banda, en el cas del tipus de caché  4-word 2-way set associative, veiem que tal i com es deia en la part teòrica, els tipus de mapeig associatius no segueixen un ordre en la caché, és a dir que un determinat bloc es pot col·locar en qualsevol posició

de la caché. Tal i com es deia, aquest tipus de caché és un intermig entre el mapeig directe i el mapeig totalment associatiu. En particular, a mesura que el processador executava el programa, es podia veure que les adreces s'anaven afegint a la caché, tot i que es va reservar un index per guardar unes adreces d'instruccions, tal i com feia la caché de mapeig directe.

Per últim, tenim que el tipus de caché 4-word fully associative no segueix l'ordre de les instruccions que hi ha al codi, sinó que segueixen l'ordre de l'execució del programa que fa el processador. És a dir, per cada vegada que s'havia d'accedir a un nou bloc ( que no fos als 4 blocs corresponents a l'índex), es carregaven 4 blocs més en l'índex següent, de forma que no es carreguen posteriorment adreces en índexos anteriors.
A continuació veiem la taula que resumeix les tasses d'encert i d'errades:

1- Memòria Caché de dades

| Tipus de Mapeig | Write-through | | | Write-back | | |
|---|---|---|---|---|---|---|
| | Hit Rate | Hits | Misses | Hit Rate | Hits | Misses |
| 4-word direct mapped | 0,76 | 19 | 6 | 0,76 | 19 | 6 |
| 4-word 2-way set associative | 0,76 | 19 | 6 | 0,76 | 19 | 6 |
| 4-word fully associative | 0,76 | 19 | 6 | 0,76 | 19 | 6 |

2- Memòria Caché d'instruccions

| Tipus de Mapeig | Write-through | | | Write-back | | |
|---|---|---|---|---|---|---|
| | Hit Rate | Hits | Misses | Hit Rate | Hits | Misses |
| 4-word direct mapped | 0,9103 | 142 | 14 | 0,9103 | 142 | 14 |
| 4-word 2-way set associative | 0,9103 | 142 | 14 | 0,9103 | 142 | 14 |
| 4-word fully associative | 0,9103 | 142 | 14 | 0,9103 | 142 | 14 |

Tal i com es pot veure, en tots dos tipus de caché tant el write-through com el write-back tenen el mateix nombre de hits i de misses.

## Cos de la Part II (i8085)

Executeu pas a pas el següent programa:

```
.define
num 02h
.data 0b
mat1: db 1,2
mat2: db 3,4
mat3: db 0,0
.data 40
pila: db 9,8
.org 600H
LXI H, pila
SPHL
MVI A,23
```

```
SUI 42
MVI B, num
LXI D, mat1
LXI H, mat2
suma:
call sumar
DCR B
jnz suma
NOP
HLT
sumar:
PUSH PSW
LDAX D
ADD M
STAX D
INX H
INX D
POP PSW
ret
```

**Qüestions:**

**Seleccioneu l'opció correcta i expliqueu els motius de la vostra selecció. Si no s'indiquen els motius**
**es considerarà la resposta com INCORRECTA.**

**1.- L'adreçament de la instrucció LXI és:**

- **a) directe**
- **b) indirecte**
- **c) immediat**
- **d) implícit**

**2.- Quina instrucció guarda el PC a la Pila?**

- **a) PUSH PC**

**b) POP PC**

**c) CALL**

**d) MOV PC, M**

En el cas de la qüestió 1, la resposta correcta és la c), és a dir l'adreçament de la instrucció LXI és immediat, ja que a un parell de registres de 16 bits se li carrega un valor immediat.

En el cas de la qüestió 2, la resposta correcta és c), és a dir CALL, ja que quan fem CALL, el que fa és guardar el PC per, una vegada haguem saltat a la subrutina i s'hagi acabat aquesta, es pugui tornar a on s'estava abans.

**Pregunta 1 Quin espai ocupa en memòria la subrutina sumar? Raona la resposta**

La subrutina *sumar* comença en la posició de memòria 0619h i acaba en la posició de memòria 620h, ambdues incloses. Per tant, fent la resta veiem ocupa 8 posicions de memòria

**Pregunta 2 Quants cicles triga en executar-se la subrutina sumar? Raona la resposta**

Per tal de respondre a aquesta pregunta, es farà ús del set d'instruccions de i8085. Per tant, tenim:

| **Instrucció** | **Cicles** |
|---|---|
| PUSH PSW | 12 |
| LDAX D | 7 |
| ADD M | 7 |
| STAX D | 7 |
| INX H | 6 |
| INX D | 6 |
| POP PSW | 10 |

| | |
|---|---|
| ret | 10 |
| **Total** | **65** |

Per tant, veiem que en total es triguen 65 cicles en executar-se la subrutina *sumar.*

## TASCA 1

**Dibuixeu el mapa de memòria de dades, per tots dos casos: adreçament i contingut. Indiqueu les instruccions que modifiquen les dades de la memòria. En cadascuna d'elles, indiqueu quines modificacions es produeixen. Com podeu veure, el simulador i8085 no te memòria cache. En el cas de l'exercici II, situeu la memòria de programa. Dins d'ella, localitza el sub-bloc que pertany a la subrutina 'sumar'.**

Per tal de fer aquesta part amb més comoditat, s'eliminaran totes aquelles posicions de memòria que no tinguin dades en elles. Per tant, el mapa de memòria de dades és:

| Direcció | Contingut |
|---|---|
| 00h | 1 |
| 01h | 2 |
| 02h | 3 |
| 03h | 4 |
| 28h | 9 |
| 29h | 8 |

Observem que en el programa es fa el .data 40, per tant es comprova que és la posició de memòria en decimal, no en hexadecimal. Té sentit pensar això, ja que 28h = 2x16 + 8 = 40d.

Les instruccions que modifiquen les dades de la memòria són PUSH PSW i POP PSW per tal de guardar i treure els valors corresponents a la pila; STAX D per guardar el valor de l'acumulador a la matriu corresponent i CALL per guardar el PC en la pila, per tal de poder tornar un cop acabada la subrutina.

En el cas de la memòria de programa, veiem que en el codi s'especifica .org 600h. Per tant, la memòria d'aquest programa comença en la posició de memòria 600h i acaba en l'última instrucció del programa, que és ret, que es troba en la posició de memòria 620h. Per tant, la memòria del programa es situa entre la posició de memòria 600h i la posició de memòria 620h. Tal i com hem dit abans, el sub-bloc *sumar* ocupa 8 posicions de memòria, i estan compreses entre la posició de memòria 619h i la posició de memòria 620h.

## TASCA 2

**Quines instruccions situen la pila en una posició determinada de la memòria? Situeu el començament de la pila en l'espai de memòria de dades del microprocessador. Indiqueu quines instruccions modifiquen la pila. Per cadascuna, indiqueu l'estat de la pila abans i després de la modificació? Cap a on creix la pila? Com s'emmagatzema la informació?**

Al principi, la pila comença en la posició de memòria 0000h, però després d'executar la instrucció SPHL, la pila queda guardada en la posició de memòria on apunta el parell de registres HL, que és la posició de memòria 0028h.

A continuació, es proporciona una taula de les instruccions que modifiquen la pila i el canvi que provoca cadascuna:

| Instrucció | Descripció | Canvi en la pila |
|---|---|---|
| PUSH PSW | Afegeix PSW a la pila | Augmenta en 2 bytes |
| CALL sumar | Guarda el PC en la pila | Augmenta en 1 byte |
| POP PSW | Treu el PSW de la pila | Disminueix en 2 bytes |
| RET | Torna al flux original | Disminueix en 2 bytes |
| SPHL | Carrega el contingut de HL | Augmenta en 1 byte |

Es pot observar que a mesura que el programa va avançant, els valors de la pila també es veuen modificats amb per exemple la instrucció SUI, que fa decréixer el valor d'acumulador en 42d. En tot cas, es pot veure que la pila creix cap a les adreces de memòria més properes al 0000h de la pila, ja que així és com funciona una pila. En particular, veiem que aquesta informació s'emmagatzema a partir de reservar un espai en la pila i posteriorment fer el push i el pop en la subrutina, per tal de treballar amb aquests valors.

## PART II

**Objectius:**

**Comprendre el direccionament a ports E/S.**

**El simulador permet comunicar el processador amb una sèrie de ports E/S:**

**Ports d'entrada: interruptors i teclat.**

**Ports de sortida: panell de LEDs, display 7 segments, display 15 segments.**

**Per accedir a aquests recursos, haurem d'accedir a les direccions adients dels ports.**

### Cos de la Part III

**Executeu pas a pas el següent programa:**

```
.data 30

pila:
.org 500H
        LXI H, pila
        SPHL
        call puertos
        NOP
        HLT
puertos:
        PUSH PSW
        in 04h
```

**ANI 00000001**

**out 05h**

**POP PSW**

**Ret**

## TASCA 3

**Què fa la subrutina 'puertos' ? Per això, introduïu dades amb els interruptors o amb el teclat; observeu en un port de sortida el resultat de la subrutina.**

Primer de tot, la subrutina *puertos* guarda l'acumulador i el registre d'estats amb la instrucció PUSH PSW. Després, fent ús de IN, agafa el valor de la direcció 04h i el guarda en l'acumulador.S'agafa aquest valor que s'ha dit i es fa l'operació AND amb el valor 00000001, i ho tornem a guardar en l'acumulador. Utilitzant la instrucció OUT agafem el valor que hi ha a l'acumulador i el mostrem per pantalla. Per últim, recuperem l'acumulador, el registre d'estats i el PC.

## PART III

**Dissenyeu un programa assembler que representi en un display 7 segments els números del 0 al 5. Els números s'introduiran a partir del teclat. A més, ha de permetre l'opció d'esborrar el display; per això, en prémer la lletra 'c', es produirà un CLEAR del display.**

**Propostes de millora:**

**El simulador 8085 té una part de la memòria de dades que implementa una 'Pantalla de text'. Modifiqueu el programa per escriure la lletra que vulgueu a la pantalla.**

**Al menú "Opciones" -> "de interrupciones" podeu habilitar que es generi una interrupció TRAP en prémer qualsevol tecla del teclat. Això produirà el salt de l'execució del progrtama a la subrutina que hi hagi a la direcció de memòria 0024h. Aprofiteu aquesta possibilitat per reproduir el funcionament continu del teclat i la pantalla.**

El codi és:

```
.data 30h                               ; Guardem tots els nombre del 0 al 5
        zero:           db 01110111b
        un:             db 01000100b
        dos:            db 00111110b
        tres:           db 01101110b
        quatre:         db 01001101b
        cinc:           db 01101011b

.data 43h                               ; Guardem el clear, tot 0
        clear:  db 00000000b

.org 100h
        loop:                           ; Fem loop infinit
                jmp loop
        hlt

        rai:                            ; Subrutina per controlar la interrupciÛ
                in 00h                  ; Guardem en acumulador el contingut del teclat
                mov C,A                 ; Movem al registre C el contingunt del acumulador
                ldax B                  ; Movem a l'acumulador el contingut el registre BC
                out 07h                 ; Mostrem al display 7 el que hi hagi a l'acumulador
                ret

.org 24h                                ; Aquí agafem la interrupciÛ
```

```
        call rai                        ; Cridem a la subrutina
        ret
```

Creem un loop infinit i associem, a la part d'Opcions: Interrupcions, la interrupció TRAP al Teclat. D'aquesta manera, quan es produeixi una variació al teclat saltarà una interrupció.
Aquesta interrupció és a la posició de memòria 24h, on truquem a la subrutina rai. Aquesta subrutina té el gran pes del programa. Primer de tot guardem en acumulador el contingut que haguem seleccionat al teclat. Aquest contingut està escrit en codi ASCII amb la qual cosa el número 0 correspon al número 30h, el número 1 al 31h, etc. La lletra A és el codi 40h i la C el 43h. És per això que hem guardat tots els números en ordre a partir de la posició 30h i el clear a la posició 43h.
El contingut de l'acumulador el movem al registre C i tot seguit copiem el contingut del parell de registres BC a l'acumulador. Aquests dos passos ens serveixen per poder obtenir el codi necessari que correspon al display de 7 segments.
Finalment, com ja tenim en acumulador el nombre necessari per representar el número al display de 7 segments tan sols hem d'utilitzar la instrucció out i especificar en quina posició olem mostrar el número, en el nostre cas 07h.

Cal comentar que l'inici del codi, és a dir com estan definits els nombres, té l'explicació donada al vídeo de la pràctica 6. En particular, pel display es representen els nombres de la forma: 0abcdefg. De tal forma que cada bit representa un "palet" d'un nombre del display. Per tant, tal i com es va explicar al vídeo, per aquest motiu es defineixen així els nombres.

La proposta de millora és la següent:

```
.org 100h
        ei                                      ; Enable interruption
        lxi H, 500h                             ; Stack pointer en 500h
        sphl
        lxi H, e000h                    ; Associem la pantalla de text al registre HL

        loop:                                   ; Fem loop infinit
                jmp loop
        hlt
```

```
        rai:                                    ; Subrutina per controlar la interrupciÛ
                in 00h                  ; Guardem en acumulador el contingut del teclat
                mov M,A                 ; Movem al registre HL el contingunt del acumulador
                inx H                           ; Incrementem H
                ei
                ret

.org 24h                                        ; AquÌ agafem la interrupciÛ
        call rai                                ; Cridem a la subrutina
        ret
```

La idea d'aquest codi és molt similar a l'anterior. Es fa un bucle infinit i es captura la interrupció en la direcció 24h. La diferència és que ara el nostre output es la pantalla de text, amb la direcció e000h en comptes del display de 7 segments.

Com a comentari, esmentar que l'explicació dels codis que es troba en aquest document és la mateixa que es troba en el fitxer on s'explica l'algorisme.

## CONCLUSIONS

Com a conclusions d'aquesta pràctica tenim que s'ha pogut aprendre a interpretar el funcionament de la memòria RAM estàtica, més coneguda com la memòria caché. En particular, mitjançant l'ús del simulador Ripes, s'ha pogut observar com es comporta la caché a mesura que avança el programa, segons el seu tipus.

Per altra banda, s'ha pogut assolir uns coneixements més avançats de les eines que proporciona el simulador i8085, com són els displays de 7 segments i les pantalles de text, així com el funcionament de la pila i l'espai que poden ocupar les subrutines, entre d'altres.